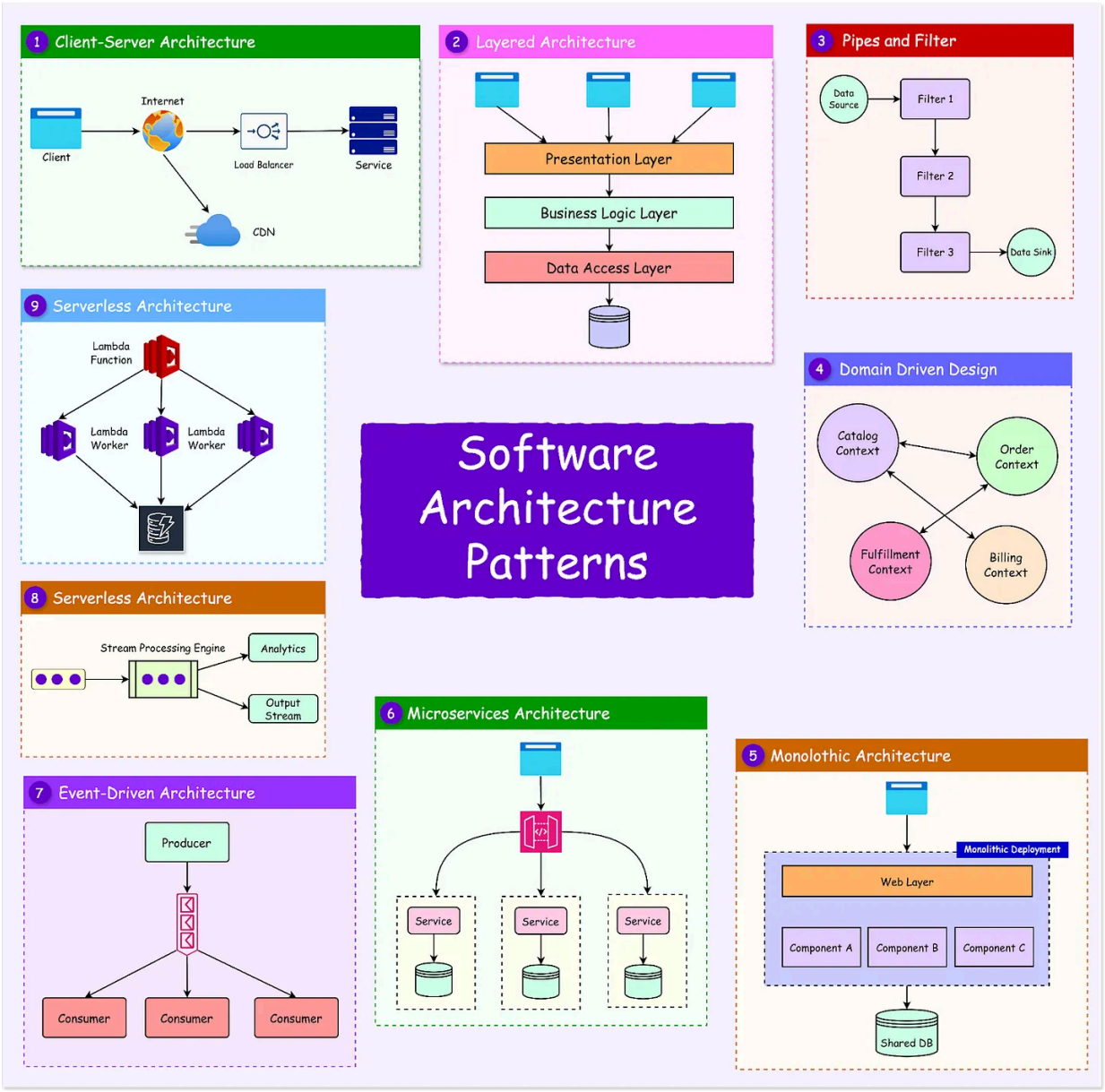
Software Architecture Patterns

Ref: https://blog.bytebytego.com/p/software-architecture-patterns

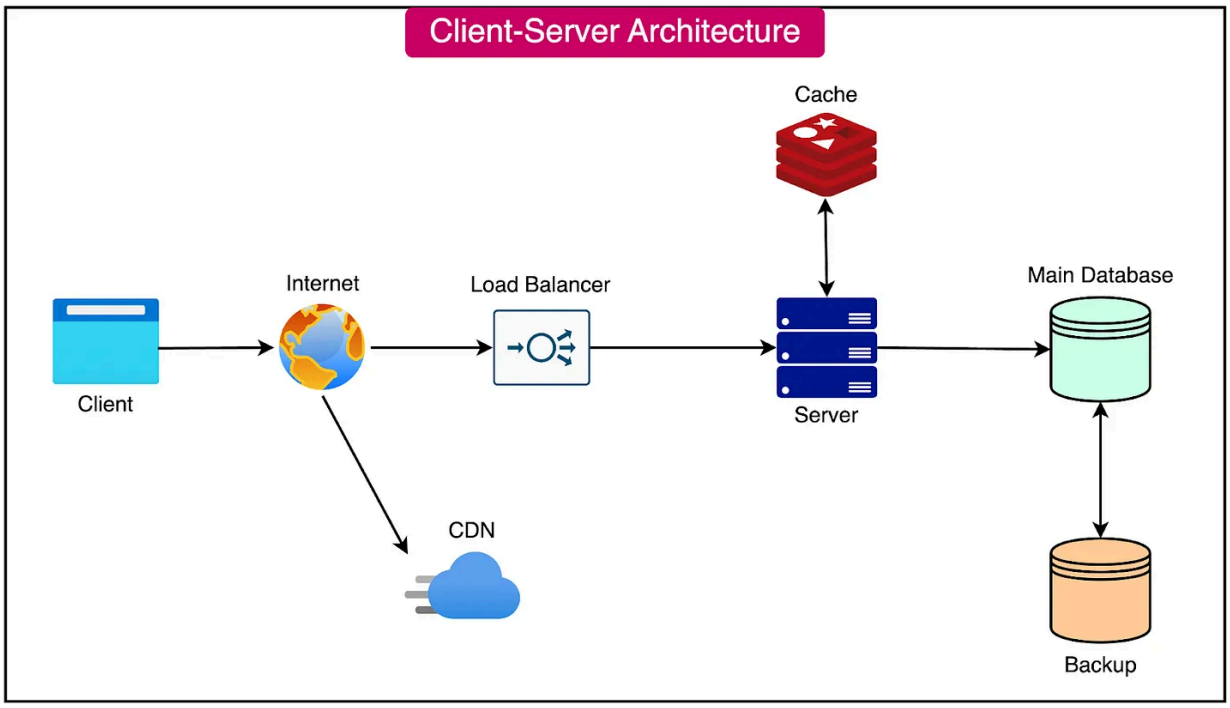
Architectural patterns offer a systematic approach to addressing recurring design issues.

In essence, **architectural patterns are reusable approaches to building software that address common design challenges.** These patterns capture the core design structures of various systems and software elements, allowing them to be reused across different projects and scenarios.



1. Client-Server Pattern

Client-server architecture is a widely used model for network communication, where a client (user or application) sends requests to a server, and the server responds with the requested data or service. This architecture can be implemented on a single machine or across different machines connected through a network.

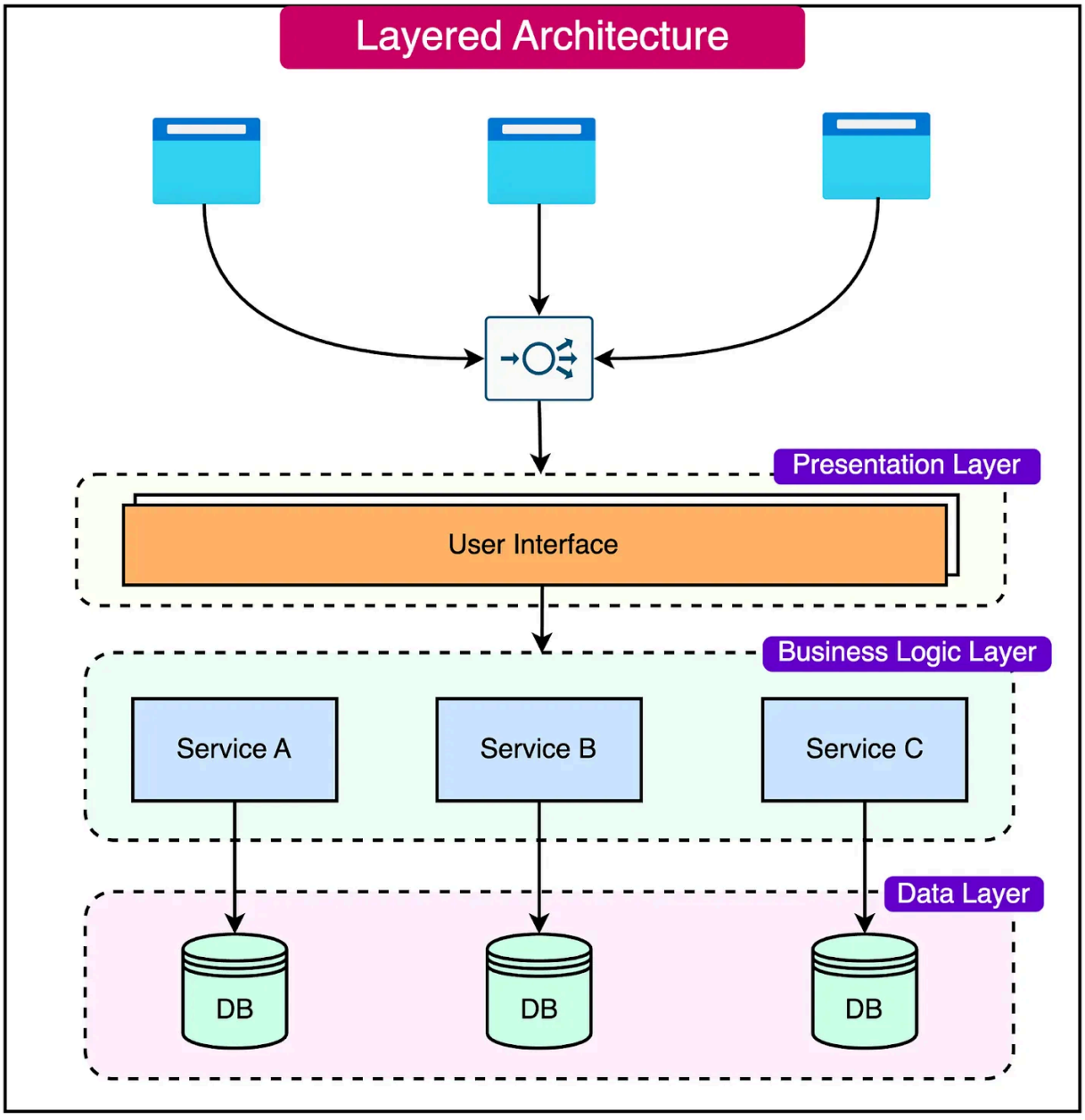


- **Client:** The client is responsible for initiating communication with the server. It sends requests to the server, specifying the desired data or service. The client can be a user interacting with an application or an application itself.
- **Server:** The server listens for incoming requests from clients. Upon receiving a request, the server processes it and returns the appropriate response to the client. The server is responsible for managing and providing access to resources and data.

2. Layered Architecture

Layered architecture is a common approach to designing complex software systems by breaking them into distinct layers, each responsible for a specific set of functionality. This architectural

pattern helps organize code, making the system easier to maintain and modify over time.



Presentation Layer

The **presentation layer** is responsible for displaying information to the user and collecting input. It encompasses the user interface and

other components that directly interact with the user.

Key responsibilities:

- **User Interface:** This is what users see and interact with, such as buttons, text boxes, and menus. It determines how the application looks and feels to the user, ensuring a smooth and intuitive interaction.
- **User Input Handling:** This layer handles user input, such as capturing data entered into forms, responding to button clicks, and validating user input to ensure data integrity.
- **Data Presentation:** The presentation layer is responsible for presenting data to the user in a meaningful and visually appealing way. It may involve formatting data, creating charts or graphs, and organizing information for easy consumption.

Business Logic Layer

The **business logic layer** is responsible for implementing the business rules of the application.

Key responsibilities:

- **Business Rules Implementation**: The business logic layer encapsulates the business rules and processes of the application.
- **Data Processing:** This layer is responsible for processing and transforming data received from the presentation layer or the data access layer. It may involve performing calculations, applying algorithms, or executing complex business logic.

- **Workflow Management:** The business logic layer manages the workflow of the application, determining the sequence of steps and actions required to complete a specific task or process.

Data Access Layer

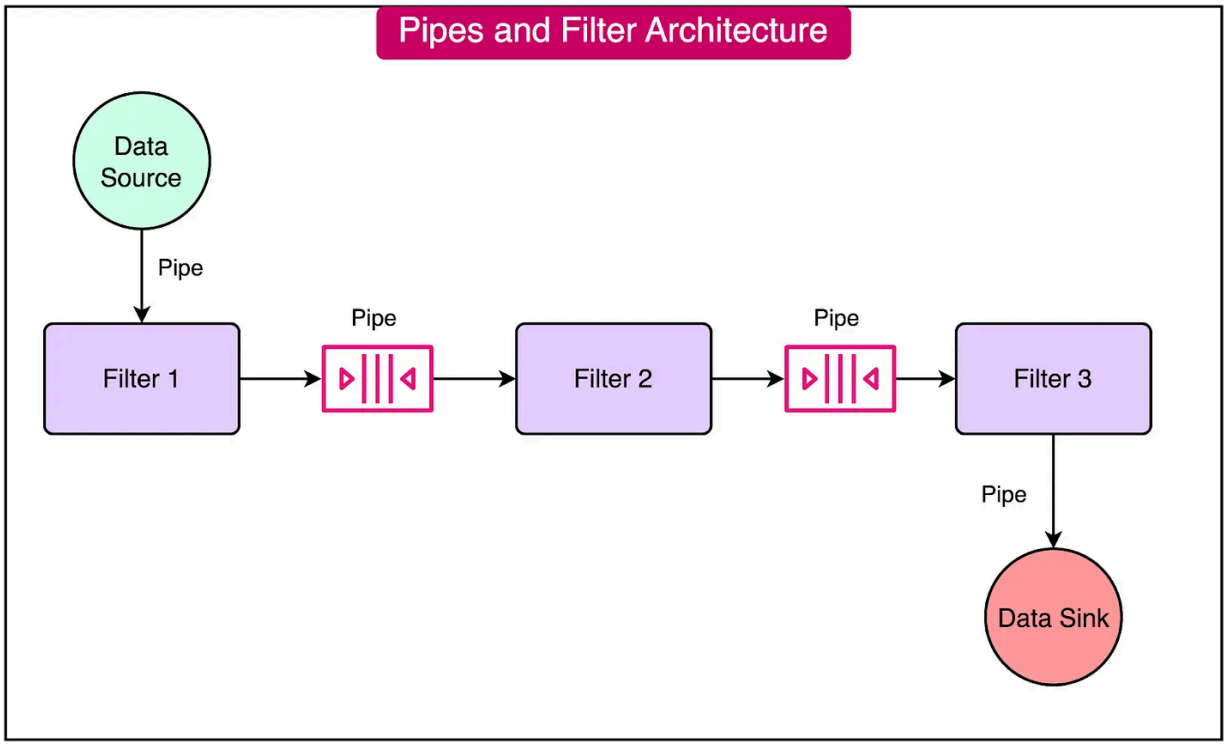
The **data access layer** is critical to the application functionality as it facilitates data persistence and retrieval. This is the application layer that interacts with databases or other external data sources.

Key responsibilities:

- **Data Retrieval:** The data access layer is responsible for retrieving data from the database or other data sources. It provides methods or APIs to query the database and fetch the required data.
- **Data Persistence:** This layer makes sure that changes made to the data are saved back to the database. It may involve executing database transactions, handling concurrency, and ensuring data integrity.
- **Data Mapping:** The data access layer often includes data mapping functionality, which maps the data retrieved from the database to the objects or entities used in the application.

3. Pipes and Filters Pattern

The **pipe and filter architectural pattern** enables software systems to process data by separating processing tasks into independent components. This architecture is particularly beneficial for systems that need to handle large amounts of data.



Key components:

Data source

The **data source** serves as the starting point of the pipeline.

It is responsible for receiving the input data and delivering it to the subsequent components in the pipeline. The data source can be any entity that provides data, such as a file, a database, or an external system.

Piple

Pipes are the connectors that link the components together.

They serve the crucial role of transferring and buffering data between the components, ensuring a smooth flow of data from an

upstream component to a downstream one.

Filter

Filters are self-contained data processing units that perform specific transformation functions on the data.

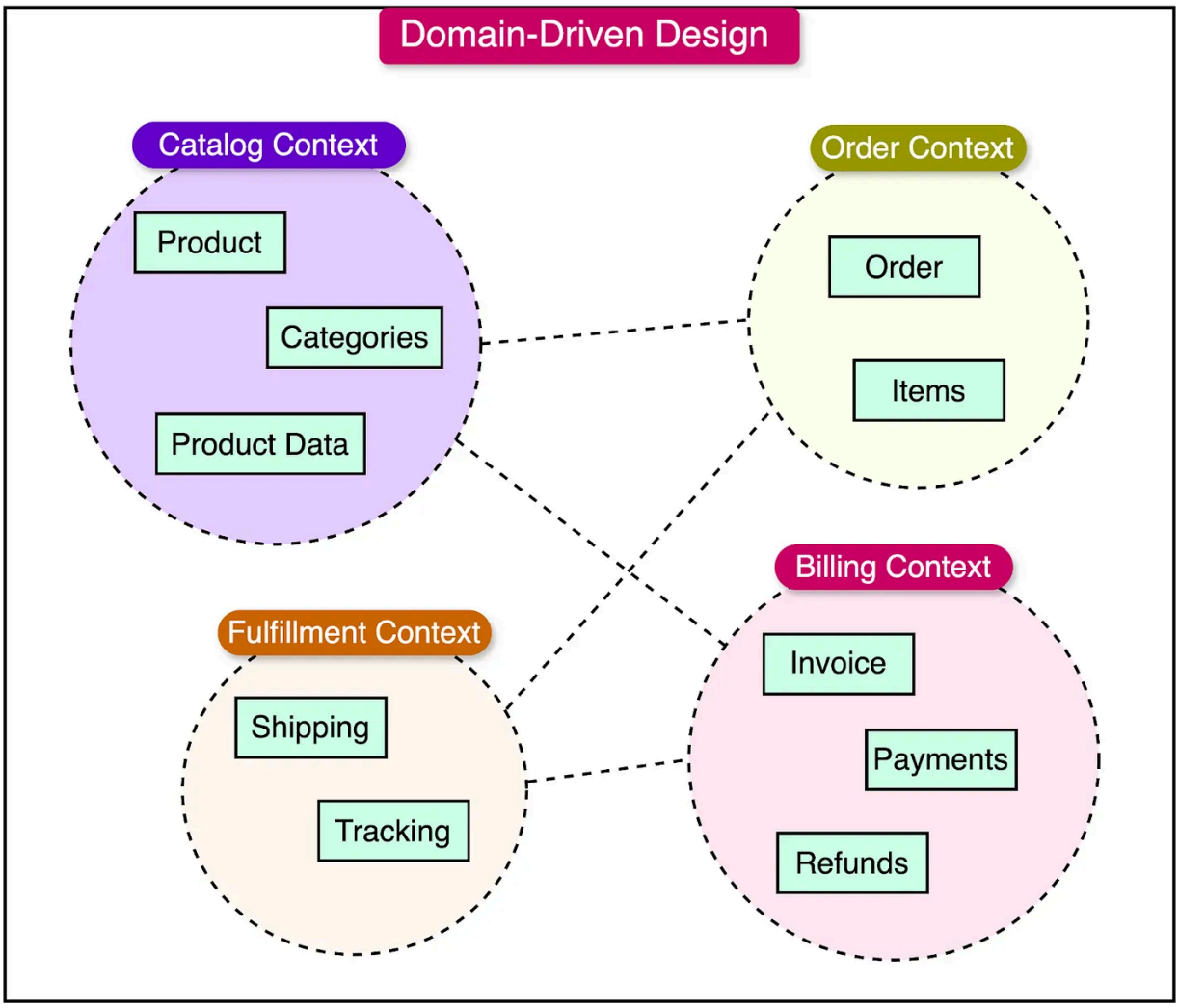
Each **filter** receives data from an upstream pipe, applies its transformation logic, and sends the transformed data to the downstream pipe. Filters are designed to be independent and reusable, allowing for modular and flexible data processing.

Data sink

The **data sink** serves as the endpoint of the pipeline.

It receives the processed data at the end of the pipeline and serves as the application's output. The data sink can be any entity that consumes or stores the processed data, such as a file, a database, or an external system.

4. Domain-Driven Design (DDD)



Domain-Driven Design (DDD) is not a typical architectural pattern. It is more of an approach to software design that emphasizes the importance of understanding and modeling the domain or problem space of a project.

DDD encourages developers to prioritize business logic and domain knowledge when designing software systems.

DDD introduces several key concepts that help in structuring the system:

- **Bounded Contexts:** Bounded Contexts are distinct areas within the software system that **encapsulate a specific domain or subdomain**. Each bounded context has its language, concepts, and rules, allowing for a clear separation of concerns and enabling teams to work independently on different parts of the system.
- **Aggregates:** Aggregates are clusters of related entities and value objects that are treated as a single unit within a Bounded Context. **They define consistency boundaries and ensure that the integrity of the domain is maintained. Aggregates help manage complexity and enforce business rules within the system.**
- **Domain Services:** Domain Services encapsulate domain-specific logic that doesn't naturally fit within any particular entity or value object. They provide a way to capture complex business operations and maintain a clear separation between the domain model and the technical implementation.

Advantages of Domain-Driven Design

- **Alignment with Business Requirements:** By focusing on the domain and involving domain experts throughout the development process, **DDD ensures that the software system accurately reflects the business requirements and meets the needs of the stakeholders.**
- **Improved Communication:** The domain model serves as a shared language between developers and domain experts, facilitating effective communication and reducing

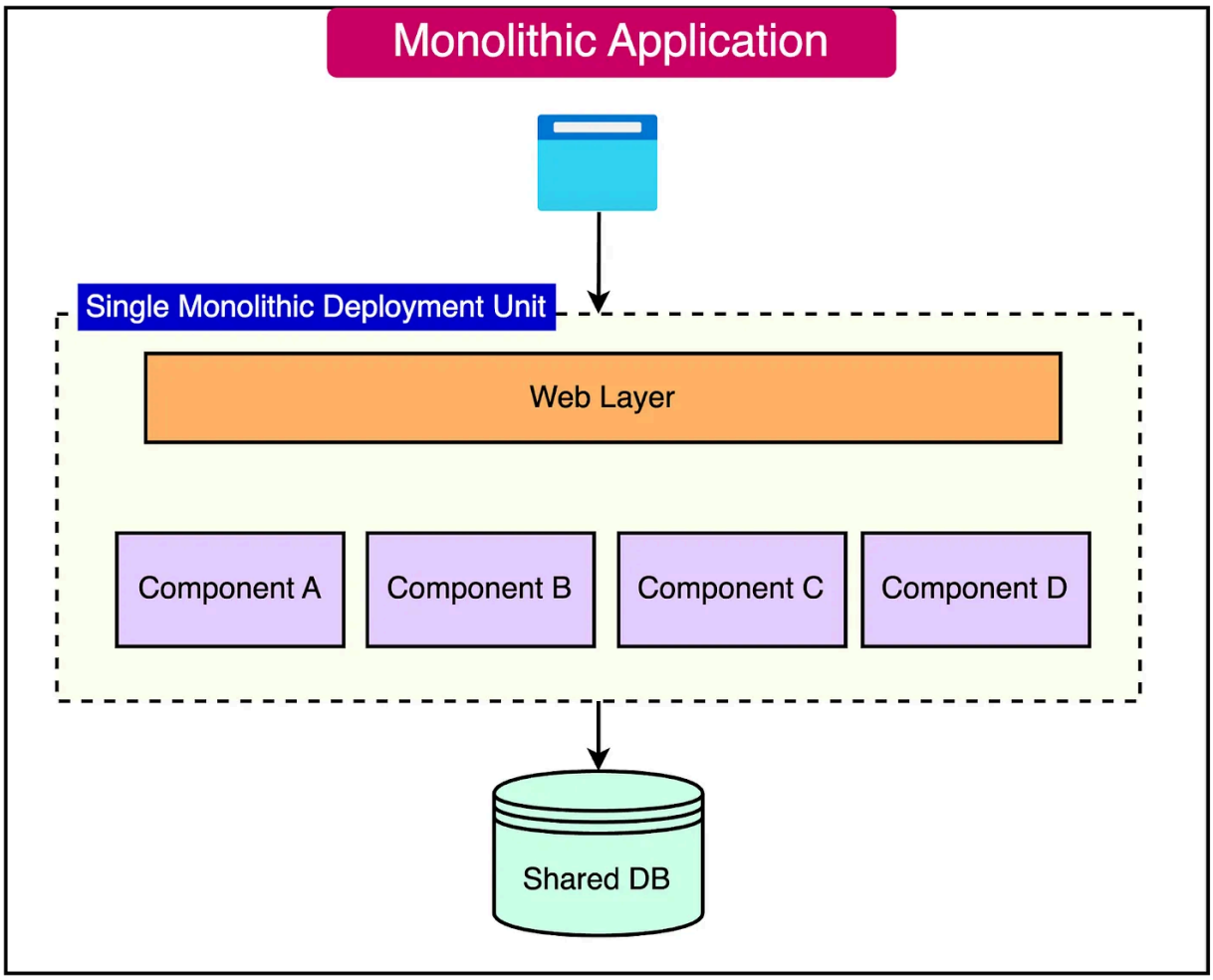
misunderstandings. It provides a common vocabulary and understanding of the problem space.

- **Modular and Maintainable Architecture:** The use of **Bounded Contexts** and **Aggregates** promotes a modular and maintainable architecture. **It allows for clear separation of concerns, enabling teams to work independently and making the system easier to understand, modify, and extend over time.**

5. Monolithic Architecture

Monolithic architecture is a software design style that has been widely used for decades.

It **involves building an application as a single, cohesive unit rather than breaking it down into smaller, individual components.** In this architecture, all the code and dependencies are packaged together, allowing the application to be deployed and run on a single server.



The key characteristics of monolithic architecture include:

- **Single, Self-Contained Unit:** In a monolithic architecture, the entire application is built as a single, self-contained unit. All the components, modules, and dependencies are tightly coupled and packaged.
- **Unified Deployment:** The application is deployed as a single unit, typically on a single server. This simplifies the deployment process, as there is no need to manage multiple deployments for different components.

Advantages of Monolithic Architecture

- **Simplicity:** One of the biggest advantages of monolithic architecture is **its simplicity**. With all the components contained within a single unit, there are fewer moving parts to manage. This simplifies the development, testing, and deployment processes.
- **Ease of Maintenance and Debugging:** Monolithic applications are easier to maintain and debug compared to distributed systems. *Since everything is located in one place, it is easier to identify and fix issues without the need to navigate through multiple components.*
- **Straightforward Development:** Developing a monolithic application is often more straightforward than developing a distributed system. Developers can focus on building the application as a whole, without worrying about the complexities of integrating and coordinating multiple components.

Disadvantages of Monolithic Architecture

- **Limited Scalability:** Monolithic applications can be challenging to scale. Since everything runs on a single server, the application's performance is limited by the server's capacity. Vertical scaling is the main option but can be costly in the long run.
- **Difficulty in Adopting New Technologies:** Monolithic applications can be resistant to change, making it difficult to adopt new technologies or programming languages. Since all the components are tightly coupled, updating a single component

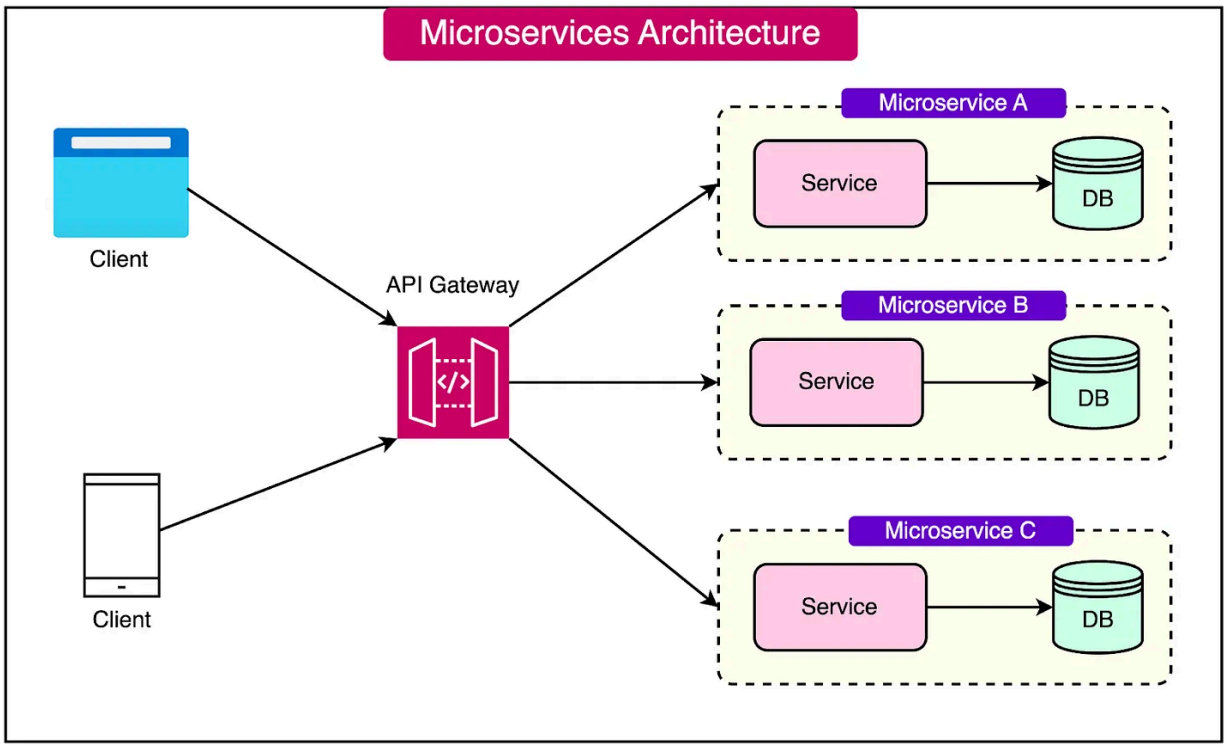
can potentially break the entire application, requiring extensive testing and modifications.

- **Tight Coupling:** Monolithic architectures tend to have tight coupling between components. This means that changes made to one part of the application can have unintended consequences on other parts. This tight coupling can make the application more brittle and harder to maintain over time.

6. Microservices Architecture

Microservices architecture is a software design approach that involves building applications as a collection of small, independent services that communicate with each other over a network.

Each service focuses on a specific business capability and can be developed, deployed, and scaled independently of other services in the system.



Advantages of Microservice Architecture

- **Improved Scalability:** In a microservices architecture, each service can be scaled independently based on specific requirements. This makes it easier to handle traffic spikes or changes in demand without affecting the entire system.
- **Increased Flexibility:** Developers can modify or add new services without impacting other parts of the system. This results in faster development and deployment of new features, as well as the ability to adapt to changing business needs.
- **Technology Diversity: Microservices** allow for the use of different technologies and programming languages for each service. This way, the teams can choose the best tools for the specific requirements of each service.

Challenges of Microservice Architecture

- **Service Communication:** Managing communication between services can be complex, especially at scale. Services need to be able to discover each other and communicate efficiently, requiring careful design and implementation of communication protocols and APIs.
- **Data Management:** In a **microservices architecture**, each service should have its data store to ensure autonomy and avoid affecting other services. This can lead to increased complexity in data management and synchronization across services.
- **Distributed System Complexity:** Microservices introduce the challenges of a distributed system, such as network latency, partial failures, and the need for distributed transactions. Developers must design and implement strategies to handle these complexities effectively.

Best Practices of Microservice Architecture

The developers should follow some best practices for designing and implementing microservices:

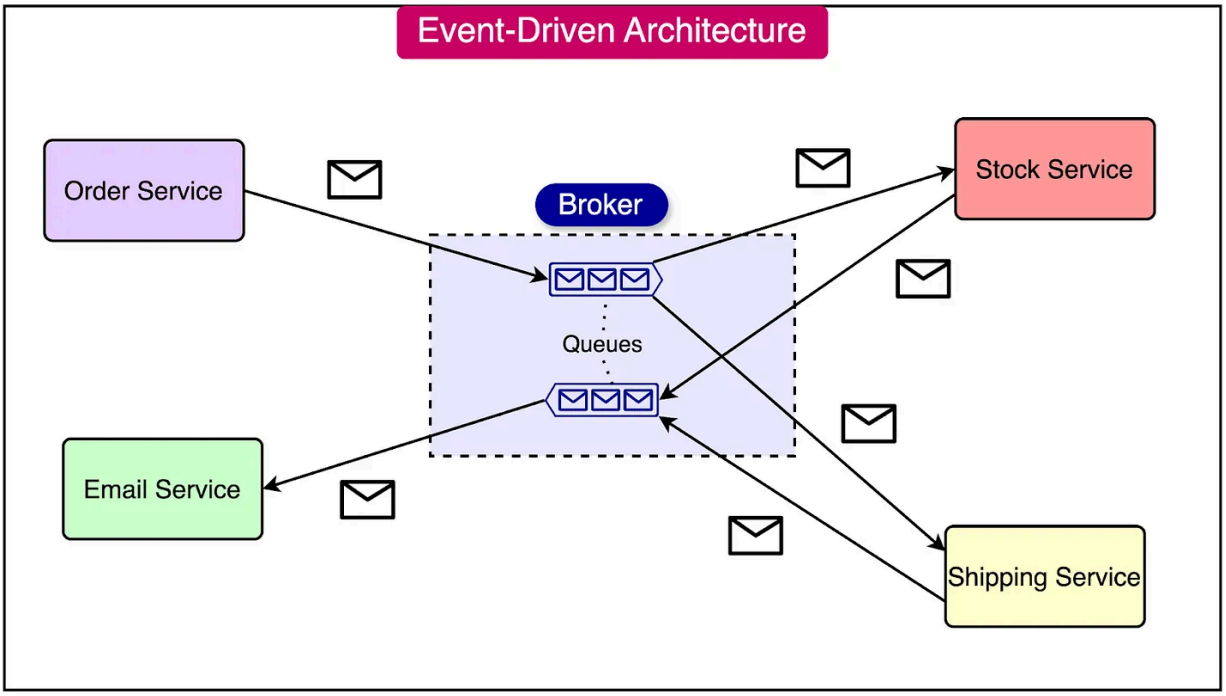
- **Design Services Around Business Capabilities:** Use Domain-Driven Design (DDD) principles to define bounded contexts and design microservices based on business capabilities. This ensures a clear separation of concerns and promotes service autonomy.
- **Loose Coupling and High Cohesion:** Design services that are loosely coupled and highly cohesive, with clear boundaries and

well-defined interfaces. This allows for independent development, deployment, and scaling of services.

- **Containerization:** Use containerization technologies like Docker to package and deploy each service as a separate container. This simplifies the scaling and deployment of individual services.
- **Monitoring and Management:** Implement effective monitoring and management tools to ensure the smooth operation of the system and quickly detect and resolve issues.
- **Continuous Integration and Deployment (CI/CD):** Implement CI/CD pipelines to automate testing and deployment of microservices. This enables faster and more reliable releases.
- **Resilience Patterns:** Design for failure by implementing resilience patterns such as circuit breakers, retries, timeouts, and fallbacks. These patterns help handle temporary failures and degraded services gracefully.
- **12-Factor Approach:** Follow the 12-Factor App methodology, which provides a set of best practices for building scalable and maintainable **microservices**.

7. Event-Driven Architecture

Event-driven architecture (EDA) is a software design paradigm that facilitates fast and efficient communication between different components or services within a system. In this architecture, components communicate through events rather than direct requests or responses.



Key concepts:

- **Events:** Events are notifications or signals that indicate the occurrence of something of interest within the system. They can originate from various sources, such as user actions (e.g., clicking a button) or system-generated notifications (e.g., data updates or errors). Events encapsulate relevant information about the occurrence and are used to trigger actions or reactions in other components.
- **Event Sources (Producers):** Event sources, also known as producers, are components or services within the system that generate events based on specific conditions or actions within the domain. When an event occurs, the event source publishes it to a central event broker or event bus.
- **Event Consumers (Subscribers):** Event consumers, also referred to as subscribers, are components or services that have

an interest in certain types of events. They subscribe to the events they are interested in and receive them from the event broker or event bus. *Consumers react to the received events by performing specific actions or triggering further processing based on the event data.*

- **Event Broker (or Event Bus):** The event broker, also known as the event bus, is a middleware component that acts as a central hub for managing the routing and delivery of events from producers to consumers. The event broker decouples event producers from event consumers, allowing for a more flexible and scalable architecture. It also ensures that events are efficiently distributed to the appropriate consumers based on their subscriptions.

Advantages of Event-Driven Architecture

- **Decoupling of Components:** In EDA, components communicate through events rather than direct requests, which reduces their dependence on each other. This decoupling makes it easier to modify or update individual components without impacting other parts of the system.
- **Scalability:** Event-driven architecture enables excellent scalability. Since events are broadcast to multiple components of the system, large amounts of data and transactions can be processed in parallel. This parallel processing capability allows the system to handle high traffic and demand spikes effectively, as the workload can be distributed across multiple event consumers.

- **Asynchronous Processing:** EDA supports the asynchronous processing of events, allowing event producers and consumers to operate independently without blocking each other. This asynchronous nature enables better resource utilization and improved performance, as components can continue processing events without waiting for responses from other components.

Challenges of Event-Driven Architecture

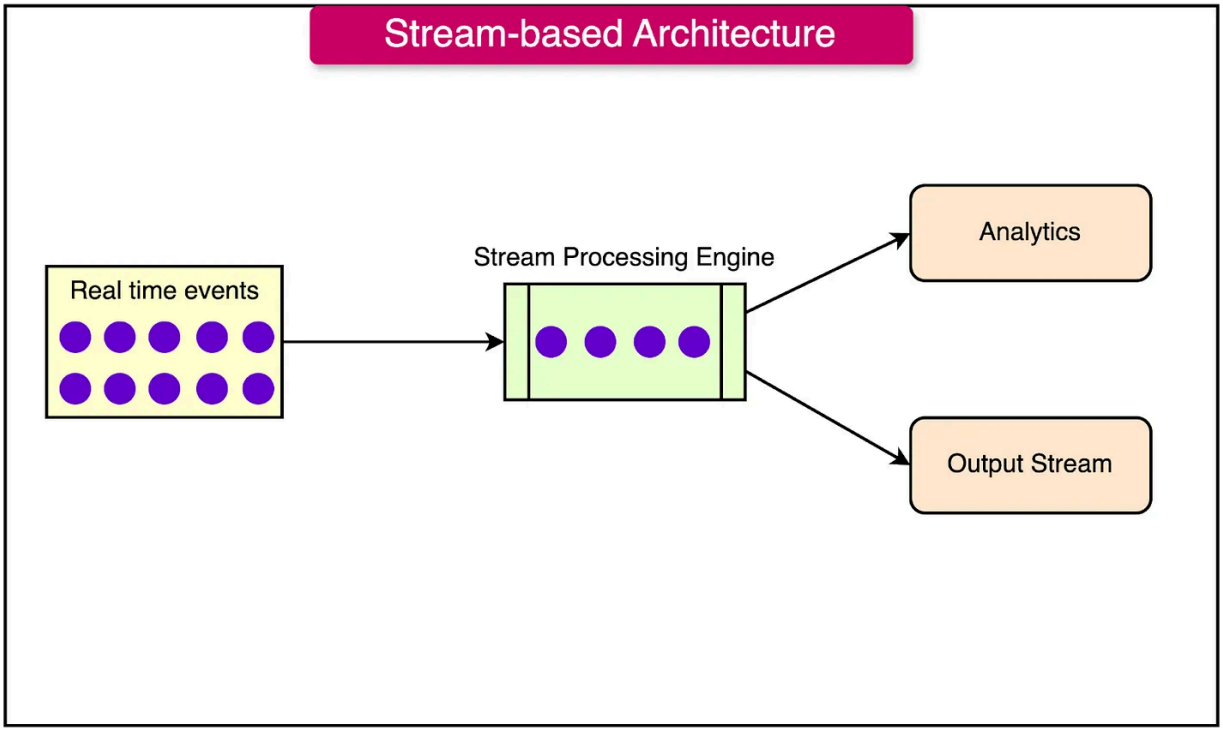
- **Complexity Management:** As events can be generated and consumed by multiple components, tracking and debugging issues can be more difficult compared to traditional request-response architectures. **Proper monitoring, logging, and tracing mechanisms need to be in place to manage and troubleshoot event-driven systems.**
- **Event Ordering:** Since events are generated and processed asynchronously, there is a risk of events being processed out of order, which can lead to data inconsistencies or calculation errors. **Mechanisms such as event sequencing, timestamps, or event sourcing may be necessary to maintain event ordering and data consistency.**
- **Error Handling and Recovery:** Since events are processed asynchronously, errors may not be immediately visible, and recovering from failures may require additional mechanisms such as event replays or compensating actions. **Robust error handling and recovery strategies need to be implemented to ensure the reliability and resilience of the system.**
- **Event Schema Evolution:** As the system evolves over time, the structure and content of events may need to change. Managing

event schema evolution can be challenging, as it requires coordination and compatibility between event producers and consumers. **Proper versioning, backward compatibility, and migration strategies need to be in place to handle event schema changes without disrupting the system.**

8. Stream-based Architecture

Stream-based architecture is a design approach that focuses on the continuous processing of data as it flows through a system.

At the core of this architecture are data streams, which are sequences of data records (events or messages) that are produced, transmitted, and consumed in real-time or near real-time.



The key principles:

- **Event-Driven Processing:** Stream-based systems process data as it is generated, rather than in batches. **This event-driven approach enables real-time processing and allows systems to respond to data changes with minimal latency.**
- **Continuous Data Flow:** Data is continuously flowing through the system, with each data record being processed as it arrives. **This continuous flow of data enables real-time analytics, pattern detection, and timely decision-making.**

Advantages of Stream-based Architecture

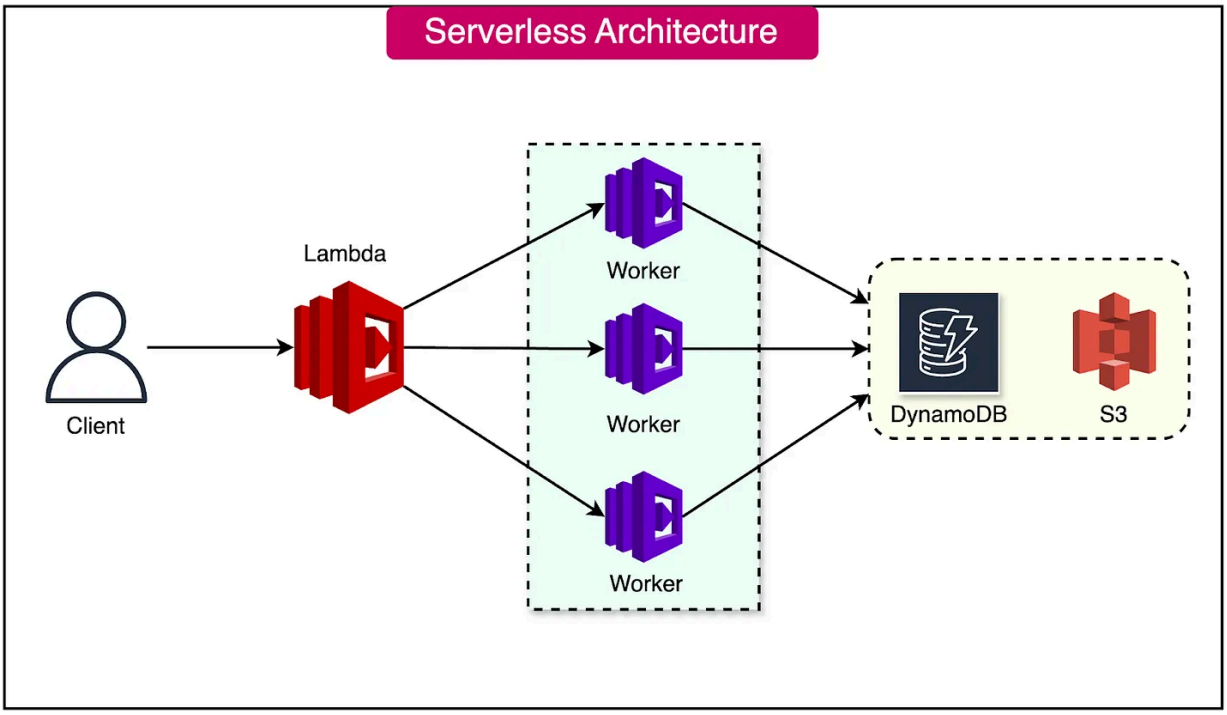
- **Real-Time Processing:** Stream-based systems enable real-time processing of data, allowing for immediate insights and timely decision-making. This is particularly valuable in scenarios where low-latency responses are critical, such as fraud detection, real-time recommendations, or IoT applications.
- **Scalability and Elasticity:** Stream-based architectures are designed to handle large volumes of data and can scale horizontally to accommodate increasing data throughput. The system can dynamically adjust its processing capacity based on the incoming data load.
    - **Scalability:** Ability to handle increased workload by adding resources. Resources are added or removed manually in response to workload changes.
    - **Elasticity**: Dynamic adjustment of resources based on demand fluctuations. Resources are automatically scaled up or down based on demand.

- **Flexibility and Adaptability:** Stream-based systems are highly flexible and can adapt to changing data sources, processing requirements, and business needs. New data sources can be easily integrated, and processing logic can be modified or extended without disrupting the overall system.
- **Fault Tolerance and Resilience:** Stream-based architectures often incorporate fault tolerance mechanisms, such as data replication, checkpointing, and exactly-once processing guarantees. **These mechanisms ensure that the system can recover from failures and maintain data integrity, even in the face of system crashes or network disruptions.**

9. Serverless Architecture

Serverless architecture is a cloud computing execution model that abstracts away the complexity of server management, allowing developers to focus on writing code and deploying individual functions or services.

In this model, the cloud provider dynamically manages the allocation and provisioning of servers, freeing developers from the burden of infrastructure management.



Key concepts:

- **Event-Driven:** Functions, also known as serverless components, are triggered by specific events such as HTTP requests, database changes, file uploads, or scheduled tasks. These events act as the driving force behind the execution of serverless functions.
- **Auto-Scaling: Serverless architecture automatically scales to match the exact demand, seamlessly adjusting the number of function instances based on the incoming workload**. Whether there are a few requests per day or thousands per second, the system dynamically adapts without requiring manual intervention.
- **Pay-Per-Use: Billing in serverless architecture is based on the actual resources consumed by the function execution, rather than pre-allocated server instances.** This means that you only

pay for the compute time and resources used during the execution of your functions, leading to cost efficiency and optimized resource utilization.

Summary

Let’s summarize the key learnings in brief:

- **Architectural patterns** are reusable approaches to building software that address common design challenges.
- **Client-server architecture** is a widely used model for network communication, where a client sends requests to a server, and the server responds with the requested data or service
- **Layered architecture** is a common approach to designing complex software systems by breaking them into distinct layers, each responsible for a specific set of functionality.
- **Pipe and filter architecture** is a design pattern that enables software systems to process data by separating processing tasks into independent components.
- The core principle of **DDD** is to gain a deep understanding of the domain in which the software operates.
- **Monolithic architecture** involves building an application as a single, cohesive unit rather than breaking it down into smaller, individual components.
- **Microservices architecture** is a software design approach that involves building applications as a collection of small, independent services that communicate with each other over a network.

- **Event-driven architecture (EDA)** is a software design approach that facilitates fast and efficient communication between different components using events.
- **Stream-based architecture** is a design approach that focuses on the continuous processing of data as it flows through a system.
- **Serverless architecture** is a cloud computing execution model that abstracts away the complexity of server management, allowing developers to focus on writing code and deploying individual functions or services.